

פרויקט — מנתח קבצי דוחות

מבוא

ברוכים הבאים לאפליקציית #C המלאה הראשונה שלכם.

בשלושת הימים האחרונים, למדתם את יסודות #C: טיפוסים, בקרת זרימה, לולאות, enums, פונקציות, מערכים והעברה בהפניה. היום ומחר, תשתמשו בכל המושגים הללו כדי לבנות מערכת עיבוד נתונים אמיתית.

פרויקט זה תוכנן במכוון כדי לאתגר את ההבנה שלכם. תיתקלו בבעיות שידרשו מכם:

- לחשוב בקפידה על טיפוסים נתונים והמרות
- לבצע דיבאג לשגיאות ניתוח
- לחקור תיעוד
- לקבל החלטות לגבי עיצוב פונקציות
- לטפל במקרי קצה

זה צפוי ומעודד.

אתם לא צפויים לדעת הכל מיד. מפתחים מקצועיים מבילים זמן רב בקריאת תיעוד, חקירת שגיאות ובדיקת ההנחות שלהם. פרויקט זה נותן לכם תרגול בדיוק בזה.

חשיבה נכונה:

- אם התוכנית שלכם קורסת בזמן ניתוח נתונים, חקרו מה השתבש
- אם הפלט שלכם לא תואם את הדוגמאות הצפויות, חקרו את טיפוס הנתונים שאתם משתמשים בהם
- אם אתם לא בטוחים כיצד לגשת לאבן דרך, פרקו אותה לשלבים קטנים יותר על נייר תחילה
- קראו הודעות שגיאה בקפידה — הן אומרות לכם בדיוק מה לא בסדר

יש לכם את כל הכלים שאתם צריכים. עכשיו הגיע הזמן לבנות משהו אמיתי.

הגבלות טכניות — חלק מהאתגר

⚠️ אין להשתמש ב-`List<T>` בפרויקט זה.

אחד האתגרים המרכזיים שהצבנו בפניכם הוא ניהול נתונים באמצעות מערכים בלבד (`int[]`, `string[]`), `double[]` וכו'.

משמעות הדבר:

- עליכם לנהל בעצמכם את מספר הרשומות התקינות
- עליכם לסנכרן מספר מערכים מקבילים
- עליכם לחשוב על גבולות המערך ולמנוע חריגה מהגודל המקסימלי
- עליכם להעביר מערכים לפונקציות ולהבין כיצד הם משתנים

למה? כי זה מחזק את ההבנה שלכם ב-Value Types מול Reference Types, Pass by Reference, וניהול מצב ידני — מושגים שלמדתם השבוע

רמז: השתמשו בקבוע כמו `const int MAX_REPORTS = 100`; כדי להגדיר את גודל המערכים.

תיאור המערכת

אתם בונים **מנתח קבצי דוחות** — אפליקציית עיבוד נתונים בצד השרת.

המערכת מקבלת קובץ טקסט המכיל דוחות שטח יומיים ממספר יחידות. כל דוח הוא שורה אחת בקובץ. כל שורה מכילה מספר פיסות מידע מופרדות בפסיקים.

חלק מהדוחות בקובץ תקינים. חלקם לא תקינים בגלל בעיות פורמט, נתונים חסרים או ערכים שגויים.

האפליקציה שלכם חייבת:

1. לקרוא את כל הדוחות מהקובץ
2. לנתח כל שורה ולחלץ את המידע
3. לאמת כל דוח לפי כללים ספציפיים
4. לדלג על דוחות לא תקינים
5. לנתח את הדוחות התקינים ולייצר סטטיסטיקות
6. להציג תוצאות לטרמינל

האפליקציה היא **כלי שורת פקודה**. אין לה ממשק גרפי. כל הקלט מגיע מקובץ. כל הפלט הולך לטרמינל.

נושא חדש — אתגר למידה עצמית

פרויקט זה דורש מכם לקרוא נתונים מקובץ טקסט. זה לא נלמד במפורש בימים 1–3.

זה מכוון.

חלק מלהיות מפתח הוא ללמוד מושגים חדשים באופן עצמאי על ידי קריאת תיעוד וניסויים.

מה אתם צריכים ללמוד

לפני שאתם מתחילים לכתוב קוד, חקרו את הדברים הבאים:

- כיצד לקרוא את כל השורות מקובץ טקסט ב-C#
- איזה סוג של מבנה נתונים מחזיק מספר שורות טקסט
- כיצד לעבוד עם נתיבי קבצים (מוחלט מול יחסי)
- מה קורה אם הקובץ לא קיים
- כיצד לבדוק אם קובץ קיים לפני ניסיון לקרוא אותו

משאבים

תיעוד רשמי של Microsoft:

- [File and Stream I/O](#)
- [File.ReadAllLines method](#)

מדריכים ידידותיים למתחילים:

- [C# File Handling \(W3Schools\)](#)

על מה להתמקד:

- המתודה `File.ReadAllLines()`
- ה-`System.IO` namespace
- איזה טיפוס `File.ReadAllLines()` מחזיר
- כיצד לטפל במקרה שבו הקובץ לא קיים

אל תחפשו פתרונות מלאים. קראו את התיעוד כדי להבין את המושג, ואז החילו אותו בעצמכם.

מפרט קובץ הקלט

האפליקציה שלכם תקרא קובץ בשם `reports.txt`.

כל שורה בקובץ מייצגת דוח אחד. הפורמט הוא:

UnitName,ReportType,Priority,Score,Status

תיאורי שדות

שדה	טיפוס	תיאור
UnitName	string	שם היחידה שהגישה את הדוח (לדוגמה, "Alpha", "Bravo")
ReportType	string	סוג הדוח — חייב להיות אחד מ: "Collect", "Analyze", "Recon", "Intel"
Priority	int	רמת עדיפות מ-1 עד 5 (1 = הנמוכה ביותר, 5 = הגבוהה ביותר)
Score	double	ציון איכות — מספר עשרוני בין 0.0 ל-100.0
Status	string	סטטוס נוכחי — חייב להיות אחד מ: "Pending", "Approved", "Rejected"

ערכים תקינים

- **UnitName**: כל מחרוזת לא ריקה (לאחר הסרת רווחים מובילים/נגררים)
- **ReportType**: חייב להיות אחד מ: **Collect**, **Analyze**, **Recon**, **Intel** (כל וריאצית אותיות מקובלת)
- **Priority**: חייב להיות מספר שלם בין 1 ל-5 (כולל)
- **Score**: חייב להיות מספר עשרוני בין 0.0 ל-100.0 (כולל)
- **Status**: חייב להיות אחד מ: **Pending**, **Approved**, **Rejected** (כל וריאצית אותיות מקובלת)

רשומות לא תקינות

רשומה היא לא תקינה אם:

- יש לה פחות מ-5 שדות (נתונים חסרים)
- יש לה יותר מ-5 שדות (נתונים עודפים)
- שדה ה-**Priority** אינו מספר שלם תקין
- ערך ה-**Priority** קטן מ-1 או גדול מ-5
- שדה ה-**Score** אינו מספר עשרוני תקין
- ערך ה-**Score** קטן מ-0.0 או גדול מ-100.0
- ה-**ReportType** אינו אחד מארבעת הערכים המותרים
- ה-**Status** אינו אחד משלושת הערכים המותרים

יש לדלג על רשומות לא תקינות — הן לא מופיעות בשום סטטיסטיקה או ניתוח.

קובץ קלט לדוגמה

צרו קובץ בשם `reports.txt` עם התוכן הבא:

Alpha,Collect,3,87.5,Approved

Bravo,Analyze,5,92.0,Approved

Charlie,Recon,2,45.3,Pending

Delta,Intel,4,78.9,Approved

Alpha,Collect,1,63.2,Rejected

Bravo,Unknown,3,88.0,Approved

Echo,Analyze,6,95.0,Approved

Foxtrot,Collect,3,invalid,Pending

Golf,Recon,2,55.7,Approved

Hotel,Intel,4,101.5,Pending

India,Collect,3,72.8,Approved

Juliet,Analyze,5,89.4,Pending

Kilo,Recon,1,34.6,Rejected

Lima,Intel,3,67.2,Approved

Mike,Collect,4,91.3,Approved

November,Analyze,2,58.9,Pending

Oscar,Recon,5,99.8,Approved

Papa,Intel,1,42.1,Rejected

Quebec,Collect,3,76.5,Approved

Romeo,Analyze,4,85.7,Approved

Sierra,Collect,2,88.5,Approved

Tango ,intel,3,77.2,Approved

Uniform,RECON,5,91.0,Pending

Victor,analyze,4,82.3,Approved

Whiskey,Collect,3, 75.5 ,Approved

Yankee,Intel,3,100.0,Approved

Zulu,Recon,1,0.0,Rejected

הערה: קובץ זה מכיל מספר רשומות מאתגרות במכון:

רשומות לא תקינות (יש לדלג עליהן):

- שורה 6: **Unknown** אינו ReportType תקין
- שורה 7: **Priority 6** מחוץ לטווח
- שורה 8: **invalid** Score אינו מספר
- שורה 10: **101.5** Score מחוץ לטווח

רשומות תקינות אך מסובכות (יש לקבל):

- שורה 21: רווחים מובילים לפני שם היחידה
- שורה 22: רווחים נגררים אחרי שם היחידה, enum באותיות קטנות
- שורה 23: enum באותיות גדולות
- שורה 24: enum באותיות קטנות
- שורה 25: רווחים סביב ערך הציון
- שורות 26-27: ערכי גבול (0.0 ו-100.0)

האפליקציה שלכם חייבת לטפל ברווחים ובווריאציות אותיות בצורה נכונה.

דרישות פונקציונליות

להלן אבני דרך מוצעות שנועדו לתת לכם מבט מלמעלה על המערכת ולהציע כיוון עבודה הגיוני. אתם לא חייבים לעבוד לפי הסדר הזה — אתם רשאים לבחור את סדר העבודה שנוח לכם.

עם זאת, שימו לב שחלק מהשליבים תלויים זה בזה (לדוגמה: קריאת קובץ חייבת לקרות לפני עיבוד הנתונים).

אבן דרך 1 — קריאת קובץ וניתוח בסיסי

מטרה: לקרוא את הקובץ ולהציג כמה שורות נמצאו.

דרישות:

1. האפליקציה שלכם חייבת לקרוא את כל השורות מ-`reports.txt`
2. אם הקובץ לא קיים, הציגו: `Error: File 'reports.txt' not found`. וצאו
3. אם הקובץ קיים אך ריק, הציגו: `Error: File is empty`. וצאו
4. אם הקובץ נקרא בהצלחה, הציגו: `File loaded: {n} lines found`.

דוגמת פלט צפוי:

אם הקובץ מכיל 20 שורות:

`File loaded: 20 lines found`

רמזי חקירה:

- איזו מתודה קוראת את כל השורות מקובץ ומחזירה אותן כמערך של מחרוזות?
- איזה namespace חייבים לייבא כדי להשתמש בפעולות קובץ?
- כיצד בודקים אם קובץ קיים לפני ניסיון לקרוא אותו?
- מה קורה אם מנסים לקרוא קובץ שלא קיים?

אבן דרך 2 — ניתוח שורה אחת

מטרה: לנתח שורה בודדת ולחלץ את כל חמשת השדות.

דרישות:

1. קחו את השורה הראשונה מהקובץ
2. פצלו אותה לשדות נפרדים
3. הציגו כל שדה בשורה נפרדת בפורמט זה:

`{Unit: {UnitName`

`{Type: {ReportType`

`{Priority: {Priority`

`{Score: {Score`

{Status: {Status

דוגמת פלט צפוי:

עבור השורה: `Alpha,Collect,3,87.5,Approved`

Unit: Alpha

Type: Collect

Priority: 3

Score: 87.5

Status: Approved

רמזי חקירה:

- כיצד לפצל מחרוזת לחלקים באמצעות מפריד ב-C-#
- איזה טיפוס המתודה `Split()` מחזירה
- כיצד לגשת לאלמנטים בודדים מהתוצאה

על מה להתמקד:

- המתודה `String.Split()`
- איזה תו להשתמש בו כמפריד (הסתכלו על פורמט הקובץ)
- אינדקס מערך — האם האלמנט הראשון מתחיל ב-0 או ב-1?
- המאפיין `Length` כדי לבדוק כמה חלקים קיבלתם

אבן דרך 3 — אימות והמרת טיפוסים נתונים

מטרה: להמיר את שדות המחרוזת לטיפוסים הנכונים שלהם ולאמת טווחים.

דרישות:

1. ניתוח את שדה ה-`Priority` ממחרוזת ל-`int`
2. ניתוח את שדה ה-`Score` ממחרוזת ל-`double`
3. בדקו ש-`Priority` נמצא בין 1 ל-5
4. בדקו ש-`Score` נמצא בין 0.0 ל-100.0
5. אם כל אימות נכשל, הציגו: `{Invalid record: {reason`

6. אם כל האימותים עוברים, הציגו: `Valid record processed`.

דוגמת פלט צפוי:

עבור שורה תקינה:

`Valid record processed`

עבור עדיפות לא תקינה:

`Invalid record: Priority out of range`

עבור פורמט ציון לא תקין:

`Invalid record: Score is not a valid number`

רמזי חקירה:

- כיצד ממירים מחרוזת למספר שלם בצורה בטוחה?
- כיצד ממירים מחרוזת ל-`double` בצורה בטוחה?
- מה קורה אם לא ניתן להמיר את המחרוזת?
- כיצד בודקים אם מספר נמצא בטווח ספציפי?

אבן דרך 4 — Enums עבור ReportType ו-Status

מטרה: להחליף השוואות מחרוזות ב-`enums`.

דרישות:

1. הכריזו על `enum` בשם `ReportType` עם ערכים: `Collect, Analyze, Recon, Intel`
2. הכריזו על `enum` בשם `ReportStatus` עם ערכים: `Pending, Approved, Rejected`
3. ניתחו את שדה ה-`ReportType` מהקובץ באמצעות `Enum.TryParse`
4. ניתחו את שדה ה-`Status` מהקובץ באמצעות `Enum.TryParse`
5. אם אחד מהניתוחים נכשל, הרשומה לא תקינה
6. הציגו את ערכי ה-`enum` המנותחים כדי לאשר שהם עובדים נכון

דוגמת פלט צפוי:

עבור השורה: `Alpha,Collect,3,87.5,Approved`

Type: Collect

Status: Approved

עבור סוג לא תקין:

Invalid record: Unknown report type

רמזי חקירה:

למדתם על enums ביום 3. עכשיו אתם צריכים להמיר מחרוזות מהקובץ לערכי enum.

לפני שאתם כותבים קוד ניתוח enum, חקרו:

- כיצד להמיר מחרוזת לערך enum בצורה בטוחה (דפוס דומה ל-`int.TryParse`)
- האם ניתוח enum רגיש לאותיות גדולות/קטנות כברירת מחדל
- כיצד להסיר רווחים מובילים ונגררים ממחרוזות

על מה להתמקד:

- המתודה `<T>.Enum.TryParse()` — שימו לב שהיא משתמשת בפרמטר טיפוס גנרי
- הפרמטר `ignoreCase` ב-`Enum.TryParse`
- המתודה `String.Trim()` — מה היא מסירה ממחרוזת

למה זה חשוב:

הנתונים לדוגמה מכילים וריאציות כמו "Intel", "intel", "INTEL", ו-"Intel". הקוד שלכם צריך לטפל בכל אלה בצורה נכונה.

אבן דרך 5 — עיבוד כל השורות וספירת תקין/לא תקין

מטרה: לעבור בלולאה על כל השורות בקובץ ולעקוב אחר כמה תקינות ולא תקינות.

דרישות:

1. צרו לולאה שמעבדת כל שורה בקובץ
2. עבור כל שורה, בצעו את כל האימותים מאבני דרך 2–4
3. שמרו מונה לרשומות תקינות
4. שמרו מונה לרשומות לא תקינות
5. בסוף, הציגו:

.Processing complete

{Valid records: {n

{Invalid records: {n

דוגמת פלט צפוי:

עבור קובץ הדוגמה עם 27 שורות (4 לא תקינות):

.Processing complete

Valid records: 23

Invalid records: 4

רמזי חקירה:

- איזה סוג של לולאה הכי טוב לאיטרציה על מערך של מחרוזות?
- כיצד עוקבים אחר מונים לאורך איטרציות הלולאה?
- האם צריך להפסיק עיבוד אם שורה אחת לא תקינה?

אבן דרך 6 — אחסון רשומות תקינות במערך

מטרה: לאחסן את כל הרשומות התקינות לניתוח מאוחר יותר.

דרישות:

1. הכריזו על מערך בגודל קבוע שיכול להכיל עד 100 דוחות
2. עבור כל רשומה תקינה, אחסנו את המידע הבא:
 - (UnitName (string
 - (ReportType (enum
 - (Priority (int
 - (Score (double
 - (Status (enum
3. עקבו אחר כמה רשומות תקינות אוחסנו
4. הציגו: `Stored {n} valid records for analysis`.

רמזי חקירה:

- אתם לא יכולים לאחסן מספר טיפוסים שונים באלמנט מערך בודד
- אתם צריכים דרך לקבץ נתונים קשורים יחד
- שקלו יצירת מערכים נפרדים לכל שדה (מערכים מקבילים)

- לדוגמה: `string[] unitNames, ReportType[] types, int[] priorities`, וכו'
- כל המערכים צריכים להיות באותו גודל ולהשתמש באותו אינדקס לנתונים קשורים

דוגמת פלט צפוי:

.Stored 16 valid records for analysis

אבן דרך 7 — חישוב סטטיסטיקות בסיסיות

מטרה: לנתח את הרשומות התקינות ולהציג סטטיסטיקות סיכום.

דרישות:

כתבו פונקציות לחישוב והצגה:

- סך הכל דוחות תקינים
- ציון ממוצע על פני כל הדוחות התקינים (מעוצב ל-2 ספרות עשרוניות)
- הציון הגבוה ביותר שנמצא בכל דוח
- הציון הנמוך ביותר שנמצא בכל דוח

הציגו את התוצאות בפורמט זה:

=== Report Statistics ===

{Total Reports: {n

{Average Score: {avg

{Highest Score: {max

{Lowest Score: {min

אבן דרך 8 — ספירת דוחות לפי סטטוס

מטרה: לספור כמה דוחות יש לכל סטטוס.

דרישות:

1. ספרו כמה דוחות יש עם סטטוס `Pending`
2. ספרו כמה דוחות יש עם סטטוס `Approved`
3. ספרו כמה דוחות יש עם סטטוס `Rejected`

4. הציגו את הספירות בפורמט זה:

=== Reports by Status ===

{Pending: {n

{Approved: {n

{Rejected: {n

דוגמת פלט צפוי:

=== Reports by Status ===

Pending: 4

Approved: 10

Rejected: 2

רמזי חקירה:

- כיצד סופרים כמה פעמים ערך enum ספציפי מופיע במערך?
- האם צריך לכתוב פונקציה אחת או שלוש פונקציות נפרדות?
- האם אפשר לכתוב פונקציה כללית שמקבלת את סטטוס היעד כפרמטר?

אבן דרך 9 — ספירת דוחות לפי סוג

מטרה: לספור כמה דוחות קיימים לכל סוג דוח.

דרישות:

הציגו את הספירה לכל סוג דוח בפורמט זה:

=== Reports by Type ===

{Collect: {n

{Analyze: {n

{Recon: {n

{Intel: {n

דוגמת פלט צפוי:

=== Reports by Type ===

Collect: 5

Analyze: 4

Recon: 4

Intel: 3

אבן דרך 10 — מציאת דוח מאושר בעדיפות הגבוהה ביותר
מטרה: למצוא את הדוח המאושר עם רמת העדיפות הגבוהה ביותר.

דרישות:

- 1. הסתכלו רק על דוחות עם סטטוס **Approved**
- 2. ביניהם, מצאו את זה עם העדיפות הגבוהה ביותר
- 3. אם יש מספר דוחות עם אותה עדיפות גבוהה ביותר, בחרו כל אחד
- 4. הציגו:

=== Highest Priority Approved Report ===

{Unit: {name

{Type: {type

{Priority: {priority

{Score: {score

דוגמת פלט צפוי:

=== Highest Priority Approved Report ===

Unit: Bravo

Type: Analyze

Priority: 5

Score: 92.0

אבן דרך 11 — חישוב ציון ממוצע לפי רמת עדיפות
מטרה: לחשב את הציון הממוצע לכל רמת עדיפות (1–5).

דרישות:

עבור כל רמת עדיפות מ-1 עד 5:

1. מצאו את כל הדוחות התקינים עם אותה עדיפות
2. חשבו את הציון הממוצע
3. הציגו את התוצאה

פורמט:

=== Average Score by Priority ===

{Priority 1: {avg

{Priority 2: {avg

{Priority 3: {avg

{Priority 4: {avg

{Priority 5: {avg

אם לא קיימים דוחות לרמת עדיפות, הציגו: **Priority {n}: No reports**

דוגמת פלט צפוי:

=== Average Score by Priority ===

Priority 1: 46.63

Priority 2: 53.30

Priority 3: 77.60

Priority 4: 85.30

Priority 5: 95.90

רמזי חקירה:

- האם צריך לכתוב פונקציה אחת או חמישה חישובים נפרדים?
- כיצד מחשבים ממוצע לתת-קבוצה של רשומות?
- מה קורה אם מנסים לחלק באפס?

אבן דרך 12 — ארגון קוד לפונקציות

מטרה: לעצב מחדש את הקוד שלכם כך ש-`Main` יהיה נקי וכל הלוגיקה תהיה בפונקציות עם שם.

דרישות:

המתודה `Main` שלכם צריכה להיות קצרה וקריאה. היא צריכה להיראות בערך כך:

```
(static void Main
```

```
}
```

```
;"string[] lines = LoadFile("reports.txt
```

```
;"if (lines == null) return
```

```
// עיבוד ואחסון רשומות תקינות
```

```
// (מבני הנתונים שלכם כאן)
```

```
;"(... ,int validCount = ProcessReports(lines
```

```
;"Console.WriteLine($"Stored {validCount} valid records for analysis
```


;(...).DisplayBasicStatistics

;(...).DisplayStatusCounts

;(...).DisplayTypeCounts

;(...).DisplayHighestPriorityApproved

;(...).DisplayAverageByPriority

{

פונקציות נדרשות:

אתם חייבים לכתוב פונקציות לאחריות הבאות. החתימות המדויקות תלויות בכם, אך לכל פונקציה צריכה להיות מטרה בודדת ברורה.

חשוב: במידה ואתם מזהים שפונקציה מסוימת עושה יותר מדבר אחד, אתם רשאים (ואפילו רצוי!) לפצל אותה לפונקציות קטנות יותר. העיקרון המנחה: **כל פונקציה צריכה לעשות דבר אחד בלבד**. פונקציות עזר נוספות מעבר לרשימה הזו מותרות ומעודדות.

1. **LoadFile** — קוראת קובץ, מטפלת בשגיאות, מחזירה מערך שורות
 2. **ProcessReports** — מעבדת שורות, מאמתת, מאחסנת במערכים, מחזירה מספר רשומות תקינות
 3. **CalculateAverage** — מקבלת מערך ציונים + מספר רשומות, מחזירה ממוצע
 4. **FindMaxScore** — מקבלת מערך ציונים + מספר רשומות, מחזירה מקסימום
 5. **FindMinScore** — מקבלת מערך ציונים + מספר רשומות, מחזירה מינימום
 6. **CountByStatus** — מקבלת מערך סטטוסים + מספר רשומות + סטטוס יעד, מחזירה ספירה
 7. **CountByType** — מקבלת מערך סוגים + מספר רשומות + סוג יעד, מחזירה ספירה
 8. **DisplayBasicStatistics** — מקבלת מערך ציונים + מספר רשומות, מחשבת ומדפיסה סטטיסטיקות
 9. **DisplayStatusCounts** — מקבלת מערך סטטוסים + מספר רשומות, סופרת ומדפיסה
 10. **DisplayTypeCounts** — מקבלת מערך סוגים + מספר רשומות, סופרת ומדפיסה
 11. **DisplayHighestPriorityApproved** — מקבלת את כל המערכים + מספר רשומות, מוצאת ומדפיסה
 12. **DisplayAverageByPriority** — מקבלת מערכי עדיפויות וציונים + מספר רשומות, מחשבת ומדפיסה
-

פונקציות נדרשות

להלן תיאור של הפונקציות המרכזיות שאתם חייבים ליישם. עבור כל פונקציה מפורט **מה** היא צריכה לעשות, אך **איך** לממש את הלוגיקה — זה עליכם לחשוב ולעצב בעצמכם. החתימות המדויקות (פרמטרים וסוג החזרה) תלויות בכם.

LoadFile

מטרה: לקרוא את כל השורות מקובץ ולטפל בשגיאות.

קלט צפוי: נתיב קובץ (string)

פלט צפוי: מערך של מחרוזות (השורות), או `null` אם לא ניתן לקרוא את הקובץ

התנהגות:

- בדקו אם הקובץ קיים
- אם לא, הציגו הודעת שגיאה והחזירו `null`
- אם הקובץ ריק, הציגו הודעת שגיאה והחזירו `null`
- אחרת, קראו את כל השורות והחזירו אותן

ProcessReports

מטרה: לעבור בלולאה על כל השורות, לאמת כל אחת ולאחסן רשומות תקינות.

קלט צפוי:

- מערך של שורות (מחרוזות)
- מערכים לאחסון נתוני רשומות תקינות

פלט צפוי: מספר הרשומות התקינות שעובדו

התנהגות:

- עבור כל שורה, פצלו אותה לשדות
- אמתו את מספר השדות
- נתחו ואמתו כל שדה
- אם תקין, אחסנו את הרשומה
- ספרו רשומות תקינות ולא תקינות
- החזירו את ספירת הרשומות התקינות

CalculateAverage

מטרה: לחשב את הממוצע של קבוצת ציונים.

קלט צפוי:

- מערך של ציונים (double)
- מספר האלמנטים התקינים לשקול

פלט צפוי: הממוצע (double)

התנהגות:

- סכמו את כל הציונים
- חלקו בספירה
- החזירו את התוצאה

FindMaxScore

מטרה: למצוא את הציון הגבוה ביותר בקבוצת רשומות.

קלט צפוי:

- מערך של ציונים (double)

פלט צפוי: הציון המקסימלי (double)

התנהגות:

- עברו בלולאה על הציונים
- עקבו אחר הערך הגבוה ביותר שנראה עד כה
- החזירו אותו

FindMinScore

מטרה: למצוא את הציון הנמוך ביותר בקבוצת רשומות.

קלט צפוי:

- מערך של ציונים (double)

פלט צפוי: הציון המינימלי (double)

התנהגות:

- עברו בלולאה על הציונים
- עקבו אחר הערך הנמוך ביותר שנראה עד כה
- החזירו אותו

CountByStatus

מטרה: לספור כמה רשומות יש עם סטטוס ספציפי.

קלט צפוי:

- מערך של ערכי סטטוס (ReportStatus)
- סטטוס היעד לספירה (ReportStatus)

פלט צפוי: כמה פעמים הסטטוס מופיע (int)

התנהגות:

- עברו בלולאה על המערך
- ספרו כמה פעמים סטטוס היעד מופיע
- החזירו את הספירה

CountByType

מטרה: לספור כמה רשומות יש עם סוג דוח ספציפי.

קלט צפוי:

- מערך של ערכי סוג (ReportType)
- סוג היעד לספירה (ReportType)

פלט צפוי: כמה פעמים הסוג מופיע (int)

התנהגות:

- עברו בלולאה על המערך
- ספרו כמה פעמים סוג היעד מופיע
- החזירו את הספירה

DisplayBasicStatistics

מטרה: לחשב ולהדפיס סטטיסטיקות בסיסיות.

קלט צפוי:

- מערך ציונים (double[])
- מספר רשומות תקינות (int)

פלט צפוי: אין (void) - מדפיסה ישירות

התנהגות:

- קוראת ל-`CalculateAverage()` כדי לחשב ממוצע
- קוראת ל-`FindMaxScore()` כדי למצוא מקסימום
- קוראת ל-`FindMinScore()` כדי למצוא מינימום
- מדפיסה את הכותרת
- מדפיסה סך הכל דוחות
- מדפיסה ציון ממוצע (מעוצב ל-2 ספרות עשרוניות)
- מדפיסה ציון גבוה ביותר
- מדפיסה ציון נמוך ביותר

DisplayStatusCounts

מטרה: לספור ולהדפיס ספירות לפי סטטוס.

קלט צפוי:

- מערך סטטוסים (ReportStatus[])
- מספר רשומות תקינות (int)

פלט צפוי: אין (void) - מדפיסה ישירות

התנהגות:

- קוראת ל-CountByStatus() עבור Pending
- קוראת ל-CountByStatus() עבור Approved
- קוראת ל-CountByStatus() עבור Rejected
- מדפיסה את הכותרת
- מדפיסה ספירה עבור כל סטטוס

DisplayTypeCounts

מטרה: לספור ולהדפיס ספירות לפי סוג דוח.

קלט צפוי:

- מערך סוגים (ReportType[])
- מספר רשומות תקינות (int)

פלט צפוי: אין (void) - מדפיסה ישירות

התנהגות:

- קוראת ל-CountByType() עבור Collect
- קוראת ל-CountByType() עבור Analyze
- קוראת ל-CountByType() עבור Recon
- קוראת ל-CountByType() עבור Intel
- מדפיסה את הכותרת
- מדפיסה ספירה עבור כל סוג דוח

DisplayHighestPriorityApproved

מטרה: למצוא ולהדפיס את הדוח המאושר בעדיפות הגבוהה ביותר.

קלט צפוי:

- מערך שמות יחידות (string[])
- מערך סוגים (ReportType[])
- מערך עדיפויות (int[])
- מערך ציונים (double[])
- מערך סטטוסים (ReportStatus[])
- מספר רשומות תקינות (int)

פלט צפוי: אין (void) - מדפיסה ישירות

התנהגות:

- עוברת בלולאה על כל הרשומות
- מסננת רק דוחות עם סטטוס Approved
- מוצאת את זה עם העדיפות הגבוהה ביותר
- מדפיסה את כל פרטיו (שם יחידה, סוג, עדיפות, ציון)

DisplayAverageByPriority

מטרה: לחשב ולהדפיס ציון ממוצע לכל רמת עדיפות (1-5).

קלט צפוי:

- מערך עדיפויות (`[]int`)
- מערך ציונים (`[]double`)
- מספר רשומות תקינות (`int`)

פלט צפוי: אין (void) - מדפיסה ישירות

התנהגות:

- עבור כל רמת עדיפות מ-1 עד 5:
 - עוברת בלולאה על כל הרשומות
 - מוצאת את כל הדוחות עם אותה עדיפות
 - מחשבת את הממוצע של הציונים שלהם
 - מדפיסה את התוצאה
 - אם אין דוחות באותה עדיפות, מדפיסה "No reports"

כללי אימות

רשומה תקינה רק אם כל התנאים הבאים מתקיימים:

1. השורה מכילה בדיוק 5 שדות כאשר מפוצלת לפי פסיק
2. שדה ה-Priority ניתן להמרה ל-`int` והוא בין 1 ל-5 (כולל)
3. שדה ה-Score ניתן להמרה ל-`double` והוא בין 0.0 ל-100.0 (כולל)
4. שדה ה-ReportType תואם אחד מארבעת ערכי ה-enum (התעלמו מאותיות גדולות/קטנות)
5. שדה ה-Status תואם אחד משלושת ערכי ה-enum (התעלמו מאותיות גדולות/קטנות)

6. שדה ה-UnitName אינו ריק לאחר הסרת רווחים

אם כל אחד מהתנאים הללו נכשל, הרשומה לא תקינה ויש לדלג עליה.

אתגרים מתקדמים (אופציונלי)

אם השלמתם את כל אבני הדרך ורוצים תרגול נוסף, נסו את ההרחבות האופציונליות הללו:

אתגר 1 — מציאת יחידה עם ציון ממוצע הגבוה ביותר

חשבו את הציון הממוצע לכל יחידה. הציגו את היחידה עם הממוצע הגבוה ביותר.

אתגר 2 — סינון דוחות לפי קריטריונים

הוסיפו פונקציה שמקבלת ציון מינימלי ועדיפות מינימלית, ומציגה רק דוחות שעומדים בשני הקריטריונים.

אתגר 3 — כתיבת תוצאות לקובץ

במקום רק להדפיס לטרמינל, כתבו את הסטטיסטיקות לקובץ חדש בשם `analysis_results.txt`.

תצטרכו לחקור:

- כיצד לכתוב טקסט לקובץ ב-`#C`
- המתודות `File.WriteAllLines` או `File.AppendAllText`

אתגר 4 — זיהוי דוחות כפולים

בדקו אם שני דוחות כלשהם יש להם בדיוק אותו `UnitName`, `ReportType` ו-`Priority`. אם נמצאו כפילויות, הציגו אזהרה.

אתגר 5 — מיון דוחות לפי ציון

הציגו את 5 הדוחות המובילים ממוינים לפי ציון (הגבוה ביותר ראשון).

תצטרכו לחקור:

- כיצד למיין מערך ב-`#C`
- כיצד לשמור על הקשר בין מערכים מקבילים בעת מיון

חשוב: אל תשתמשו בקלאסים או אובייקטים לאתגרים אלה. השתמשו רק במושגים שנלמדו בשבוע 1.

טעויות נפוצות שיש להימנע מהן

שגיאות טיפוס נתונים

- **טעות:** שימוש ב-`int` לאחסון ציונים במקום `double`
 - **תוצאה:** ערכים עשרוניים נחתכים (87.5 הופך ל-87)
 - **חקירה:** בדקו את דוגמאות הפלט הצפוי — האם הן מציגות ספרות עשרוניות?
-

שגיאות ניתוח

- **טעות:** אי בדיקה אם `int.Parse` או `double.Parse` יצליחו לפני קריאה אליהם
 - **תוצאה:** התוכנית קורסת עם `FormatException`
 - **חקירה:** אילו מתודות מאפשרות לכם לנסות לנתח מבלי לקרוס?
-

שגיאות אינדקס מערך

- **טעות:** גישה ל-`array[count]` כאשר `count` שווה ל-`array.Length`
 - **תוצאה:** `IndexOutOfRangeException`
 - **חקירה:** מה הטווח התקין של אינדקסים למערך?
-

שגיאות ניתוח Enum

- **טעות:** הנחה ש-`Enum.TryParse` אינו רגיש לאותיות גדולות/קטנות
 - **תוצאה:** ערכים תקינים מסומנים כלא תקינים
 - **חקירה:** קראו את התיעוד ל-`Enum.TryParse` — מה ההתנהגות כברירת מחדל?
-

שגיאות חישוב

- **טעות:** חלוקת שני מספרים שלמים וציפייה לתוצאה עשרונית
 - **תוצאה:** הממוצע תמיד מספר שלם
 - **חקירה:** מה קורה כאשר כותבים `sum / count` אם שניהם `int`?
-

שגיאות Scope

- **טעות:** הכרזה על מונה בתוך לולאה
 - **תוצאה:** המונה מתאפס ל-0 בכל איטרציה
 - **חקירה:** היכן צריך להכריז על משתנים אם הם צריכים להימשך לאורך איטרציות לולאה?
-

שגיאות נתיב קובץ

- **טעות:** שימוש בנתיב מוחלט שעובד רק על המכונה שלכם
 - **תוצאה:** התוכנית נכשלת כאשר רצה על מחשב אחר
 - **חקירה:** מה ההבדל בין נתיבים מוחלטים ויחסיים?
-

אסטרטגיות דיבאג

כאשר התוכנית שלכם לא עובדת כצפוי:

אסטרטגיה 1 — הדפיסו ערכי ביניים

הוסיפו הצהרות `Console.WriteLine` כדי לראות מה המשתנים שלכם מכילים בכל שלב.

דוגמה:

```
;"{Console.WriteLine($"DEBUG: fields.Length = {fields.Length}");  
;"{[Console.WriteLine($"DEBUG: Priority string = '{fields[2}");
```

הסירו הצהרות דיבאג אלה לאחר שהבעיה נפתרה.

אסטרטגיה 2 — בדקו עם שורה אחת תחילה

לפני עיבוד הקובץ כולו, בדקו את לוגיקת הניתוח והאימות שלכם על שורה ידועה בודדת.

אסטרטגיה 3 — בדקו את הודעת השגיאה

קראו את הודעת השגיאה כולה. היא אומרת לכם:

- איזה סוג של שגיאה התרחשה
- איזו שורת קוד גרמה לה
- איזה ערך היה לא תקין

אסטרטגיה 4 — אמתו הנחות

אם אתם מניחים ששדה תמיד מספר תקין, בדקו מה קורה כאשר הוא לא.

אסטרטגיה 5 — השתמשו בתיעוד

אם אתם לא בטוחים כיצד מתודה עובדת, קראו את התיעוד הרשמי. הוא כולל דוגמאות ומסביר מקרי קצה.

פלט סופי צפוי

כאשר אתם מריצים את התוכנית המושלמת שלכם עם קובץ `reports.txt` לדוגמה, הפלט צריך להיראות כך:

.File loaded: 27 lines found

.Processing complete

Valid records: 23

Invalid records: 4

.Stored 23 valid records for analysis

=== Report Statistics ===

Total Reports: 23

Average Score: 73.84

Highest Score: 100.0

Lowest Score: 0.0

=== Reports by Status ===

Pending: 5

Approved: 16

Rejected: 2

=== Reports by Type ===

Collect: 8

Analyze: 6

Recon: 5

Intel: 4

=== Highest Priority Approved Report ===

Unit: Uniform

Type: Recon

Priority: 5

Score: 91.0

=== Average Score by Priority ===

Priority 1: 31.60

Priority 2: 62.50

Priority 3: 78.13

Priority 4: 82.58

Priority 5: 92.76

הערה: הערכים המדויקים יהיו תלויים באילו רשומות תקינות וכיצד אתם מטפלים בשוויון. הפורמט והמבנה צריכים להתאים.

הנחיות הגשה

ההגשה הסופית שלכם צריכה לכלול:

1. **Program.cs** — קוד ה-C# המלא שלכם
2. **reports.txt** — קובץ הקלט שהשתמשתם בו לבדיקה
3. **output.txt** — קובץ טקסט המכיל את הפלט שהתוכנית שלכם מייצרת

דרישות איכות קוד:

- כל הפונקציות חייבות להיות בעלות שמות ברורים ותיאוריים
- שמות משתנים חייבים להיות משמעותיים (לא `x, y, temp`, אלא אם כן מתאים)
- `Main` צריך להיות קצר ולקרוא לפונקציות עם שם
- אין קוד מוערות בהגשה הסופית
- אין הצהרות הדפסת דיבאג בגרסה הסופית

הצעות ניהול זמן

זהו פרויקט של יומיים. הנה לוח זמנים מוצע:

יום 4 (רביעי)

- **09:00 – 10:00** — קריאת תיאור הפרויקט, חקירת קריאת קבצים
- **10:00 – 11:00** — אבני דרך 1–2 (קריאת קובץ וניתוח בסיסי)
- **11:00 – 12:00** — אבן דרך 3 (אימות והמרת טיפוס)
- **14:00 – 15:00** — אבן דרך 4 (enums)
- **15:00 – 16:30** — אבני דרך 5–6 (עיבוד כל השורות, אחסון רשומות)
- **16:30 – 17:30** — אבן דרך 7 (סטטיסטיקות בסיסיות)

יום 5 (חמישי)

- **09:00 – 10:30** — אבני דרך 8–9 (ספירה לפי סטטוס וסוג)
- **10:30 – 12:00** — אבני דרך 10–11 (עדיפות גבוהה ביותר, ממוצע לפי עדיפות)
- **14:00 – 16:30** — אבן דרך 12 (עיצוב מחדש לפונקציות)
- **16:30 – 17:30** — בדיקה, דיבאג, ניקיון סופי

התאימו את לוח הזמנים הזה בהתאם לקצב שלכם. חלק מהסטודנטים יתקדמו מהר יותר, אחרים לאט יותר. שניהם בסדר.

מחשבות אחרונות

פרויקט זה מאחד את כל מה שלמדתם השבוע:

- טיפוסים נתונים — int, double, string, bool, enums
- משתנים והצבות — אחסון ועדכון ערכים
- ניתוח והמרות — string ל-string, int ל-string, double ל-enum
- פונקציות — ארגון קוד לחלקים הניתנים לשימוש חוזר
- מערכים — אחסון אוספים של נתונים קשורים
- Enums — החלפת מספרים קסומים בשמות משמעותיים
- לולאות — עיבוד מספר רשומות
- ערכי החזרה — פונקציות שמחשבות ומחזירות תוצאות
- Scope — הבנה היכן משתנים חיים
- העברה לפי ערך מול העברה בהפניה — שליטה כיצד נתונים משותפים

לא תכתבו קוד מושלם בניסיון הראשון. תעשו:

- ניסיון
- פגישה בשגיאות
- חקירה
- תיקון
- למידה

כך מפתחים מקצועיים עובדים.

יש לכם את כל מה שאתם צריכים. בהצלחה.